

Nevronske mreže

Ljupčo Todorovski

Univerza v Ljubljani, Fakulteta za matematiko in fiziko
Institut Jožef Stefan, Odsek za tehnologije znanja (E8)

April 2026

Pregled predavanja

- 1 Osnovni gradniki
- 2 Usmerjene, *feed-forward*, nevronske mreže
 - Plasti in struktura
 - Enoplastni Perceptron
 - Običajne funkcije aktivacije
 - Vzratno razširjanje napake, *back propagation*
 - Kompromis med predsodkom in varianco
- 3 Konvolucijske nevronske mreže
 - Konvolucijska plast
 - Akumulacijska, *pooling*, plast
- 4 Rekurentne (*recurrent*) nevronske mreže
 - Osnovna ideja
 - Vzratno razširjanje napake skozi čas
 - Težave

Nevroni in sinapse

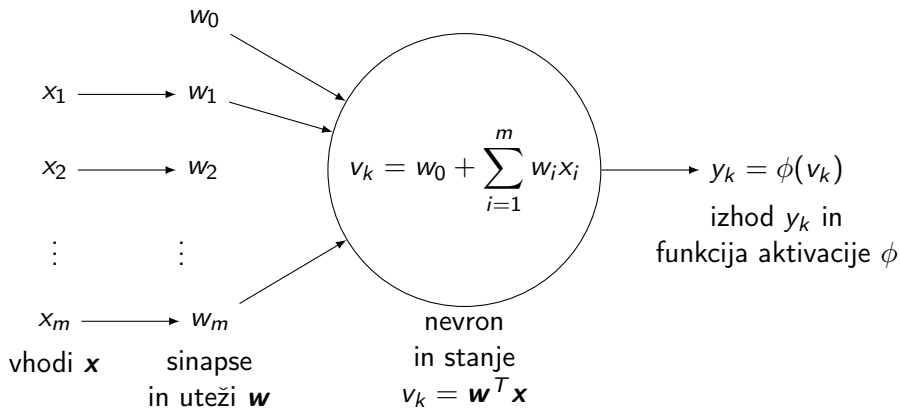
Osnovna gradnika nevronske mreže (NM)

- Nevron (angl. *neuron*): ima **stanje** $v \in \mathbb{R}$ in **izhod** $y \in \mathbb{R}$
- Sinapsa (angl. *synapse*): povezava med nevroni, ki ima **utež** $w \in \mathbb{R}$
- Sinapse določajo **strukturo** (tudi arhitekturo, topologijo) NM

Izvajanje nevronske mreže

- Podatki spremenijo stanja (in nato izhode) izbranih, vhodnih nevronov
- Spremenjeni izhodi nevronov se prenašajo po sinapsah
- Nevron, ob spremembi vhodnih sinaps, izračuna novo stanje in izhod
- Izračun se v splošnem nadaljuje, dokler se izhodi nevronov ne ustalijo
- Izračun mreže preberemo iz izhodov izbranih, izhodnih nevronov

Funkcija nevrona



Vektorska notacija: $\mathbf{x} = (1, x_1, x_2, \dots, x_m)^T$ in $\mathbf{w} = (w_0, w_1, \dots, w_m)^T$.

Nevron kot funkcija

$$N: \mathbb{R}^{m+1} \rightarrow \mathbb{R}$$

$$N: [1 \ x_1 \ x_2 \ \dots \ x_m]^T \mapsto \phi \left(w_0 + \sum_{i=1}^m w_i x_i \right) = \phi (\langle \mathbf{w}, \mathbf{x} \rangle)$$

Funkcija, ki preslika vhode nevrona v njegov izhod

- Vhod je vektor realnih števil, to so izhodi povezanih nevronov
- Če teh m števil označimo z $x_1, \dots, x_m$, je vektor $\mathbf{x} = [1 \ x_1 \ x_2 \ \dots \ x_m]^T$
- Vektor uteži $\mathbf{w} = [w_0 \ w_1 \ w_2 \ \dots \ w_m]^T$, w_0 je **utež premika** (angl. *bias*)
- $\langle \mathbf{w}, \mathbf{x} \rangle$ je skalarni produkt vektorjev $\mathbf{w}$ in $\mathbf{x}$ iz $\mathbb{R}^{m+1}$

Struktura usmerjenih nevronskih mrež

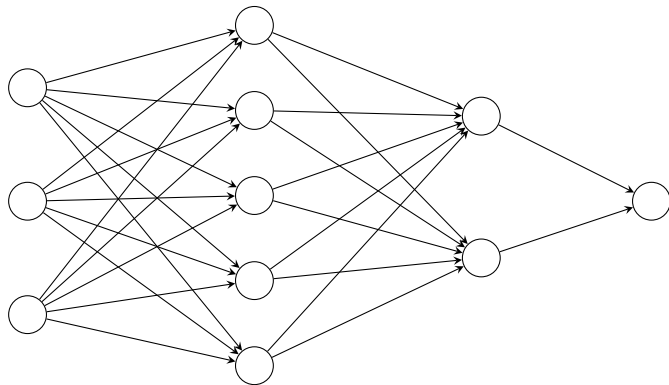
Nevroni organizirani v zaporedje plasti $\ell_0, \ell_1, \dots, \ell_{L+1}$

- Nevroni v eni plasti niso medsebojno povezani
- Vsak nevron iz ℓ_l je povezan z vsakim nevronom iz ℓ_{l+1} , $l = 0, 1, \dots, L$
- Nevroni iz dveh plasti, ki nista zaporedni, niso povezani

Poimenovanje plasti

- ℓ_0 je prva, **vhodna** plast (angl. *input layer*) **vhodnih nevronov**
- $\ell_1, \ell_2, \dots, \ell_L$ so **skrite** plasti (angl. *hidden layers*) **skritih nevronov**
- ℓ_{L+1} je zadnja, **izhodna** plast (angl. *output layer*) **izhodnih nevronov**

Primer usmerjene nevronske mreže: Dve skriti plasti



Koliko parametrov (uteži) ima ta usmerjena nevronska mreža (vključno z odmiki)?

Enostavni perceptron za binarno klasifikacijo

Dve ravni nevronov ($L = 0$)

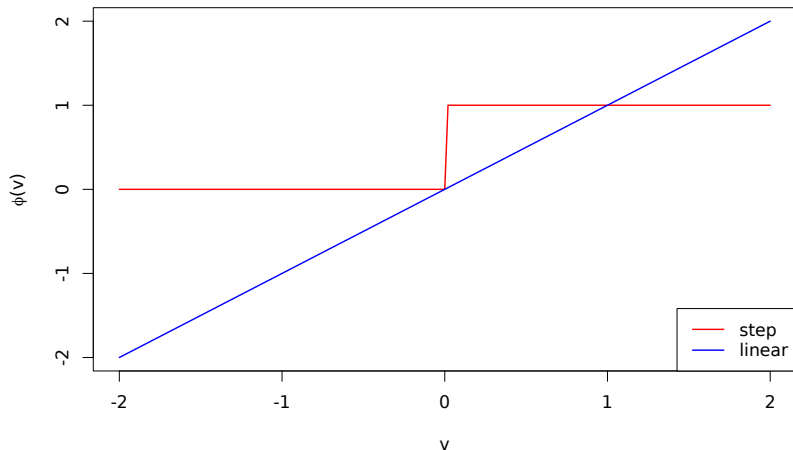
- Raven 0 vhodnih nevronov, funkcije aktivacije $\phi(v) = v$
- Raven 1 izhodnih nevronov, funkcija aktivacije **stopnica**

$$\phi_{step}(v) = \begin{cases} 0, & v \leq 0 \\ 1, & v > 0 \end{cases}$$

Še ena verzija linearnega modela za klasifikacijo

$$\hat{y} = \phi_{step}(\mathbf{w}^T \mathbf{x})$$

Običajni funkciji aktivacije v perceptronu ϕ_{step} in $\phi(v) = v$



Iterativno pravilo za učenje uteži enostavnega perceptrona

$$\Delta \mathbf{w} = \eta (y - \phi_{step}(\mathbf{w}^T \mathbf{x})) \mathbf{x}$$

- V vsaki iteraciji obravnavamo en primer $e = (\mathbf{x}, y)$
- $\Delta \mathbf{w}$ je sprememba uteži v iteraciji
- η je parameter, ki določa hitrost učenja, *learning rate*
- Začetne vrednosti $\mathbf{w}$ nastavimo naključno

Če sta razreda linearno ločljiva, algoritem konvergira (končno št. iteracij).

Pravilo delta za učenje uteži perceptrona

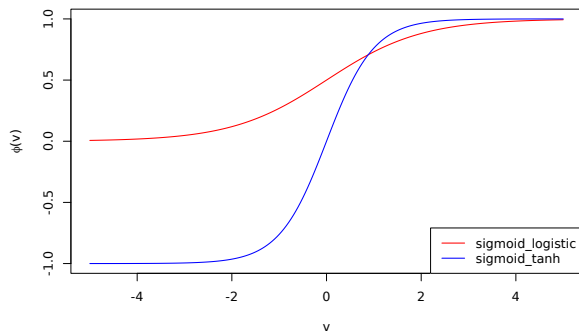
Izračun gradienta za $E = (y - \hat{y})^2$, $\hat{y} = \phi(v)$, $v = \mathbf{w}^T \mathbf{x} = \sum_k w_k x_k$

$$\begin{aligned}\frac{\partial E}{\partial w_i} &= \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_i} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \phi(v)}{\partial v} \frac{\partial v}{\partial w_i} \\ &= -2(y - \hat{y}) \phi'(v) \frac{\partial \sum_k w_k x_k}{\partial w_i} \\ &= -2(y - \phi(\mathbf{w}^T \mathbf{x})) \phi'(v) x_i\end{aligned}$$

Torej, $\Delta \mathbf{w} = \eta (y - \phi(\mathbf{w}^T \mathbf{x})) \phi'(v) \mathbf{x}$.

Sigmoidni funkciji aktivacije v usmerjenih NM

$$\phi_{\text{logistic}}(v) = \frac{1}{1 + e^{-v}}, \quad \phi_{\text{tanh}}(v) = \tanh v$$



Druge funkcije aktivacije v usmerjenih NM

Linearna, *Lin* in rektifikacijska (angl. *rectifier*), *ReLU* funkcija aktivacije

$$\phi_{Lin}(v) = v$$

$$\phi_{ReLU}(v) = \max(0, v) = \begin{cases} 0 & \text{če } v \leq 0 \\ v & \text{sicer} \end{cases}$$

Normalizacijska eksponentna funkcija, *softmax*

$$\phi_{softmax}(v_i) = \frac{e^{v_i}}{\sum_{k=1}^{n_{L+1}} e^{v_k^{(L+1)}}}$$

- Običajno jo uporabljamo v zadnji plasti usmerjene NM
- Indeks i se nanaša na obravnavani izhodni nevron
- Indeks k teče skozi izhodne nevrone v plasti ℓ_{L+1}
- Zagotovi, da je seštevek izhodov izhodnih nevronov enak 1

Običajne arhitekture usmerjenih NM

Za regresijo

- Število vhodnih nevronov $n_0 = p$, število napovednih spremenljivk
- En izhodni nevron z linearno funkcijo aktivacije

Za klasifikacijo

- Število vhodnih nevronov $n_0 = p$, število napovednih spremenljivk
- Število izhodnih nevronov $n_{L+1} = |D_Y|$, število vrednosti ciljne spr.
- V izhodni plasti normalizacijska eksponentna funkcija *softmax*
- Vsak izhodni nevron napove verjetnost ene vrednosti ciljne spr.
- Dvojiška klasifikacija izjema: zgolj en izhodni nevron, ki napove verjetnost izbrane (pozitivne) vrednosti ciljne spremenljivke

Notacija v izpeljavah in formulah

- $W^{(l+1)}$ je matrika uteži na sinapsah med ℓ_l in ℓ_{l+1}
- $w_{ji}^{(l+1)}$ je utež sinapse med j -tim nevronom plasti ℓ_l in i -tim nevronom plasti ℓ_{l+1} , hkrati element matrike $W^{(l+1)}$
- $v_i^{(l+1)}$ je stanje i -tega nevrona plasti ℓ_{l+1} , torej v poenostavljeni notaciji velja $v_i^{(l+1)} = \sum_{j=0}^{n_l} w_{ji}^{(l+1)} y_j^{(l)}$
- $y_i^{(l+1)}$ je izhod i -tega nevrona plasti ℓ_{l+1} , $y_i^{(l+1)} = \phi \left(v_i^{(l+1)} \right)$

Funkcija izgube za regresijo in njen odvod

Kvadratna napaka

$$\mathcal{L} = \frac{1}{2} \sum_i (y_i - \hat{y}_i)^2 = \frac{1}{2} \sum_i \left(y_i - y_i^{(L+1)} \right)^2,$$

ker velja $\hat{y}_i = y_i^{(L+1)}$.

Odvodi funkcije izgube po izhodih

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = -(y_i - \hat{y}_i) = - \left(y_i - y_i^{(L+1)} \right)$$

Funkcija izgube za klasifikacijo

Prečna entropija H med

- Opazovano porazdelitvijo p vrednosti ciljne spremenljivke Y
- Porazdelitvijo q vrednosti Y , ki jo napove NM, $q(v_i) = y_i^{(L+1)}$

$$\mathcal{L} = H(p, q) = - \sum_{v_i \in D_Y} p(v_i) \log_2 q(v_i) = - \sum_{v_i \in D_Y} p(v_i) \log_2 y_i^{(L+1)}$$

Za podan učni primer z vrednostjo ciljne spremenljivke $Y = v_m$

$$\mathcal{L} = -\log_2 q(v_m) = -\log_2 y_m^{(L+1)},$$

ker velja $p(v_m) = 1$ in $p(v_i) = 0$ za $\forall v_i \in D_Y \setminus \{v_m\}$.

Odvod prečne entropije po izhodih

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = \begin{cases} -\frac{1}{y_m^{(L+1)} \log 2} & \text{če } i = m \\ 0 & \text{sicer} \end{cases}$$

Posplošeno pravilo delta: Izhodna plast

$$E = \frac{1}{2} \sum_i (y_i - \hat{y}_i)^2, \hat{y}_i = y_i^{(L+1)} = \phi(v_i^{(L+1)}), v_i^{(L+1)} = \sum_j w_{ji}^{(L+1)} y_j^{(L)}$$

$$\begin{aligned} \frac{\partial E}{\partial w_{ji}^{(L+1)}} &= \frac{\partial E}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial v_i^{(L+1)}} \frac{\partial v_i^{(L+1)}}{\partial w_{ji}^{(L+1)}} \\ &= -(y_i - y_i^{(L+1)}) \phi'(v_i^{(L+1)}) \frac{\partial \sum_k w_{ki}^{(L+1)} y_k^{(L)}}{\partial w_{ji}^{(L+1)}} \\ &= -(y_i - y_i^{(L+1)}) \phi'(v_i^{(L+1)}) y_j^{(L)} \end{aligned}$$

$$\Delta w_{ji}^{(L+1)} = \eta (y_i - y_i^{(L+1)}) \phi'(v_i^{(L+1)}) y_j^{(L)}$$

Posplošeno pravilo delta: Skrite plasti

$$\frac{\partial E}{\partial w_{ji}^{(l)}} = \frac{\partial E}{\partial y_i^{(l)}} \frac{\partial y_i^{(l)}}{\partial v_i^{(l)}} \frac{\partial v_i^{(l)}}{\partial w_{ji}^{(l)}} = \frac{\partial E}{\partial y_i^{(l)}} \phi'(v_i^{(l)}) y_j^{(l-1)}$$

Trik za izračun $\partial E / \partial y_i^{(l)}$ iz $\partial E / \partial y_k^{(l+1)}$

$$\begin{aligned} \frac{\partial E}{\partial y_i^{(l)}} &= \sum_k \frac{\partial E}{\partial v_k^{(l+1)}} \frac{\partial v_k^{(l+1)}}{\partial y_i^{(l)}} \\ &= \sum_k \frac{\partial E}{\partial y_k^{(l+1)}} \frac{\partial y_k^{(l+1)}}{\partial v_k^{(l+1)}} \frac{\partial v_k^{(l+1)}}{\partial y_i^{(l)}} \\ &= \sum_k \frac{\partial E}{\partial y_k^{(l+1)}} \phi'(v_k^{(l+1)}) w_{ik}^{(l+1)} \end{aligned}$$

Posplošeno pravilo delta: Skrite plasti (nadaljevanje)

$$\Delta w_{ji}^{(l)} = \eta \phi'(v_i^{(l)}) y_j^{(l-1)} \sum_k \frac{\partial E}{\partial y_k^{(l+1)}} \phi'(v_k^{(l+1)}) w_{ik}^{(l+1)}$$

- Iteracija pravila od skrite plasti L do skrite plasti 1
- V plasti 0 upoštevamo $y_i^{(0)} = x_i$, t.j., vrednost spremenljivke X_i

Velikost in kontrola velikosti nevronskih mrež

Velikost nevronske mreže

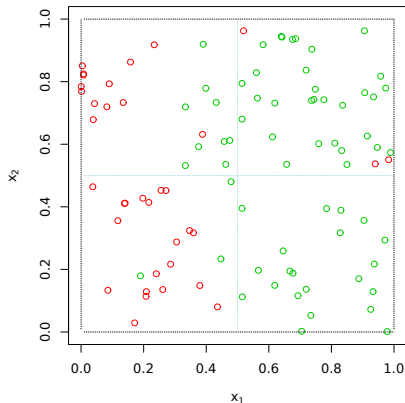
- Večanje nevronske mreže poveča število sinaps
- Torej število prostih parametrov napovednega modela
- Zato se zmanjša predsodek in poveča varianca

Kontrola velikosti NM

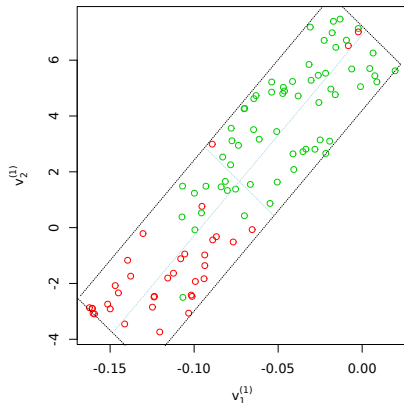
- Število skritih plasti (globina), globoke, *deep*, NM
- Število nevronov v vsaki skriti plasti

Plast skritih nevronov in nelinearnost pri klasifikaciji

Vhodna plast

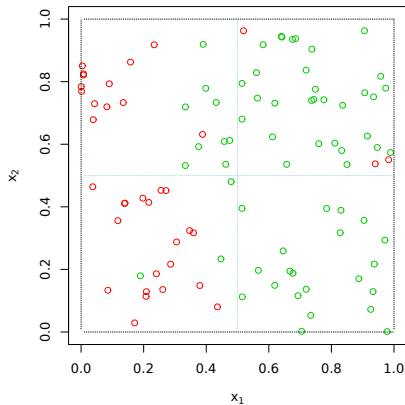


Skrita plast (stanja nevronov)

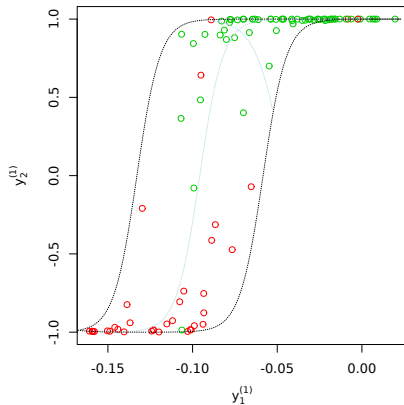


Plast skritih nevronov in nelinearnost pri klasifikaciji

Vhodna plast



Skrita plast (izhodi nevronov)



Konvolucijski filter

Notacija

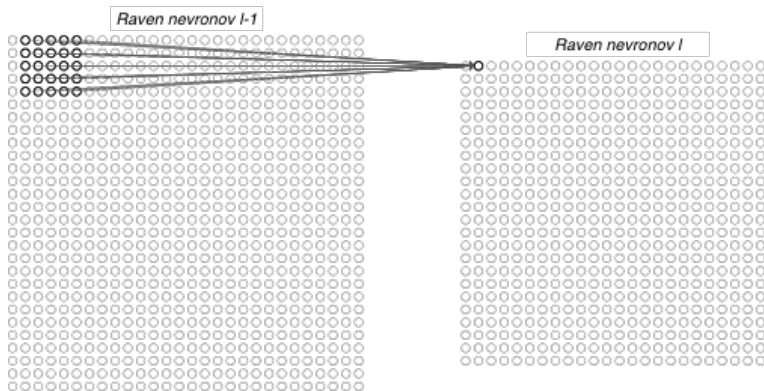
- Nevroni v plasti $l - 1$ organizirani v matriko dimenzij $x \times y$
- Uteži w_0 in **konvolucijski filter** dimenzij $c \times c$, elementi $w_{i,j}$
- Plast l je matrika nevronov dimenzij $(x - c + 1) \times (y - c + 1)$

Navzkrižna korelacija, uporabljena namesto običajne linearne obtežene vsote

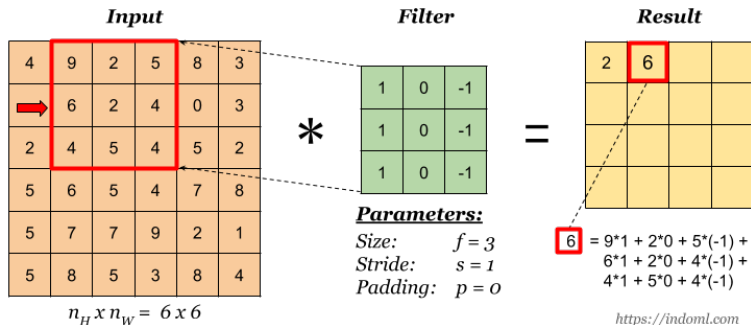
$$v_{i,j}^{(l)} = w_0 + \sum_{k=1}^c \sum_{m=1}^c w_{k,m} y_{i+k-1,j+m-1}^{(l-1)}$$

- $y_{i,j}^{(l-1)}$ in $v_{i,j}^{(l)}$ so izhodi in stanja nevronov v plasti $l - 1$ oz. l
- Običajna dimenzija konvolucijskega filtra je $c = 5$

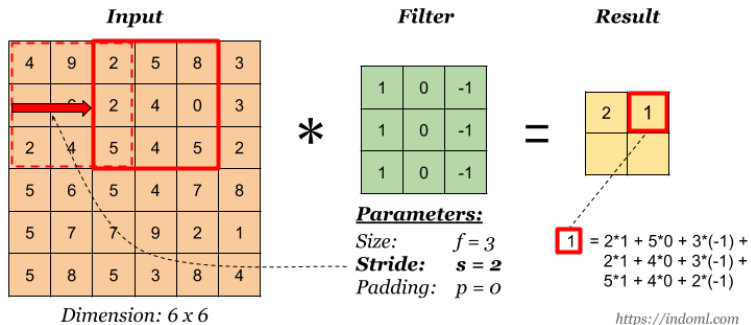
Grafični prikaz



Primer izračuna



Parameter korak, *stride*



Zmanjševanje dimenzij slike

Notacija

- Nevroni v plasti $l - 1$ organizirani v matriko dimenzij $x \times y$
- Plast l je matrika nevronov dimenzij $(x/a) \times (y/a)$
- Predpostavka: $a|x$ in $a|y$

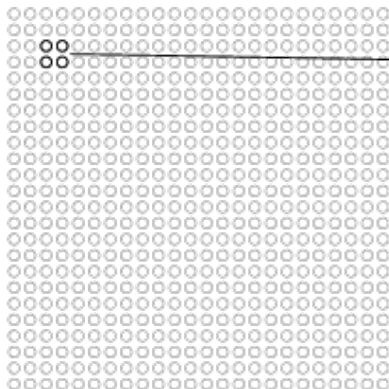
max-akumulacija, uporabljena namesto običajne linearne obtežene vsote

$$v_{i,j}^{(l)} = \max \left\{ y_{(i-1) \cdot a + 1, (j-1) \cdot a + 1}^{(l-1)}, \dots, y_{i \cdot a, j \cdot a}^{(l-1)} \right\}$$

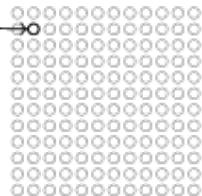
- $y_{i,j}^{(l-1)}$ in $v_{i,j}^{(l)}$ so izhodi in stanja nevronov v plasti $l - 1$ oz. l
- Običajna vrednost $a = 2$

Grafični prikaz

Raven nevronov I-1 (običajno izhodna raven konvolucije)

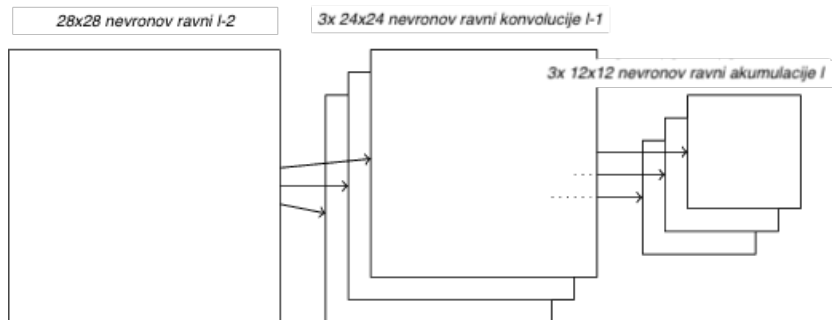


Raven nevronov I



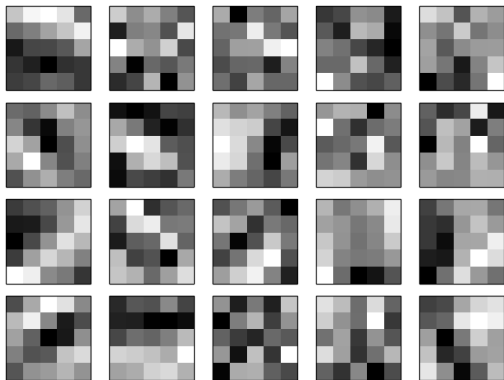
Običajna topologija: $r \times$ konvolucija $\rightarrow r \times$ akumulacija

Uporaba za klasifikacijo slik in razpoznavanje z roko napisanih števk.



Razpoznavanje na roko napisanih števk

- Klasifikacijska napaka klasičnih metod: okoli 3%
- Klasifikacijska napaka konvolucijskih NM: manj kot 0.5%

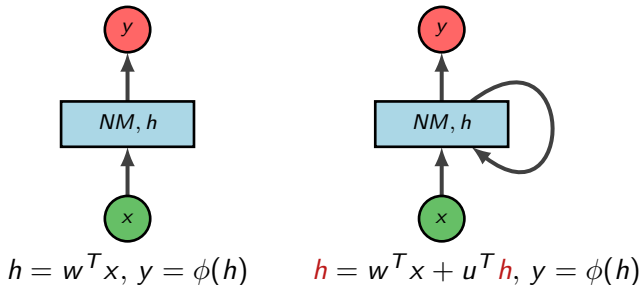


Posebnosti učenja nevronske mreže

- Vse diskretne spremenljivke je treba spremeniti v numerične
- Običajno je treba opraviti normalizacijo podatkov
- Težave z gradientno metodo in lokalnimi ekstremi
- Večkratni izračuni, stohastične metode
- Funkcije aktivacije prilagojene podatkom in problemu

Nevronski mreži dodamo zanko

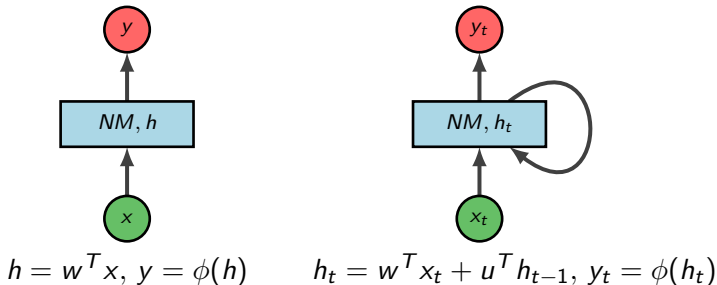
Rekurentna nevronska mreža = navadna nevronska mreža + zanka



To ni več usmerjena nevronska mreža, težave z računanjem vrednosti h .

Rešitev težave: časovni indeks t

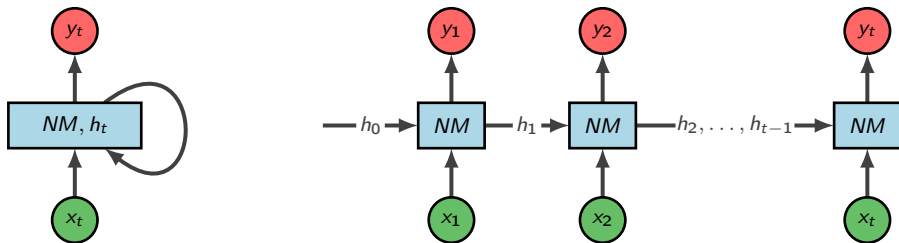
Rekurentna nevronska mreža = navadna nevronska mreža + zanka + čas



Na prvi pogled ne reši težav, a pogledjmo interpretacijo indeksa t .

Indeks pravzaprav pretvori RNM v usmerjeno NM

Rekurentna nevronska mreža = usmerjena nevronska mreža



$$h_t = w^T x_t + u^T h_{t-1}, y_t = \phi(h_t), h_0 = 0$$

Plasti ustrezajo časovnemu indeksu: vzvratno razširjanje napake skozi čas.

Vloga stanja h v RNN

Stanje h v RNN predstavlja pomnilnik

- Ker velja $h_t = f(x_t, h_{t-1})$
- Stanje h_{t-1} si lahko **zapomni** prejšnje vhode $x_1, \dots, x_{t-1}$
- In zato obravnava x_t v kontekstu prejšnjih vhodov, h_{t-1}

Splošen predpis

- $h_t = f(x_t, h_{t-1}, W^{(h)})$, $W^{(h)}$ matrika uteži NM za stanje
- $y_t = g(h_t, W^{(y)})$, $W^{(y)}$ matrika uteži NM za izhod

Posebnosti rekurentne NM pri učenju

Računanje poteka običajno

- Od leve proti desni, iteracija skozi čas $t = 1, 2, \dots, T$
- Od spodaj navzgor, iteracija skozi plasti usmerjenih NM

Vzratno razširjanje napake se zaplete

- Zaradi potovanja signala nazaj skozi čas
- Posredno odvajanje po času, v nadaljevanju

Napaka E rekurentne mreže

Pozor, napako moramo opazovati skozi čas

$$E = \frac{1}{T} \sum_{t=1}^T L(y_t, \hat{y}_t)$$

L je poljubna funkcija izgube.

Odvod napake E po utežeh $W^{(h)}$ za stanje

$$\begin{aligned}\frac{\partial E}{\partial W^{(h)}} &= \frac{1}{T} \sum_{t=1}^T \frac{\partial L(y_t, \hat{y}_t)}{\partial W^{(h)}} \\ &= \frac{1}{T} \sum_{t=1}^T \frac{\partial L(y_t, \hat{y}_t)}{\partial \hat{y}_t} \frac{\partial g(h_t, W^{(y)})}{\partial h_t} \frac{\partial h_t}{\partial W^{(h)}}\end{aligned}$$

Spomnimo se, $h_t = f(x_t, h_{t-1}, W^{(h)})$, zato

$$\frac{\partial h_t}{\partial W^{(h)}} = \frac{\partial f(x_t, h_{t-1}, W^{(h)})}{\partial W^{(h)}} + \frac{\partial f(x_t, h_{t-1}, W^{(h)})}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial W^{(h)}}$$

Vpeljimo začasne oznake a_t , b_t in c_t

$$\begin{aligned}a_t &= \frac{\partial h_t}{\partial w^{(h)}} \\b_t &= \frac{\partial f(x_t, h_{t-1}, W^{(h)})}{\partial w^{(h)}} \\c_t &= \frac{\partial f(x_t, h_{t-1}, W^{(h)})}{\partial h_{t-1}}\end{aligned}$$

Velja torej rekurzivna enačba $a_t = b_t + c_t a_{t-1}$.

Rešitev rekurzivne enačbe

$$\begin{aligned}
 a_t &= b_t + c_t a_{t-1} \\
 &= b_t + c_t (b_{t-1} + c_{t-1} a_{t-2}) \\
 &= b_t + c_t b_{t-1} + c_t c_{t-1} (b_{t-2} + c_{t-2} a_{t-3}) \\
 &= b_t + c_t b_{t-1} + c_t c_{t-1} b_{t-2} + c_t c_{t-1} c_{t-2} a_{t-3} \\
 &= \vdots
 \end{aligned}$$

$$= b_t + \sum_{i=1}^{t-1} b_i \prod_{j=i+1}^t c_j + a_0 \prod_{k=1}^t c_k$$

$$\{h_0 = 0, a_0 = 0\} = b_t + \sum_{i=1}^{t-1} b_i \prod_{j=i+1}^t c_j$$

Če zamenjamo nazaj, dobimo

$$\frac{\partial h_t}{\partial w^{(h)}} = \frac{\partial f(x_t, h_{t-1}, W^{(h)})}{\partial w^{(h)}} + \sum_{i=1}^{t-1} \frac{\partial f(x_i, h_{i-1}, W^{(h)})}{\partial w^{(h)}} \prod_{j=i+1}^t \frac{\partial f(x_j, h_{j-1}, W^{(h)})}{\partial h_{j-1}}$$

V praksi računamo vsoto zadnjih $\tau < t - 1$ členov: $i = t - \tau, \dots, t - 1$
 Zato **skrajšano** (*truncated*) vzvratno razširjanje napake skozi čas.

Težave osnovne, *vanilla* RNN

Veliko število izračunov

- Opomba: razgrnjene mreže *NM* **delijo iste uteži**
- Razgrnitev *NM* skozi čas poveča število izračunov
- Delna rešitev: manjšanje časovnega horizonta

Kratkoročni spomin

- Vpliv x_1 na h_t **skozi čas blede**, ker
- Na vsakem vmesnem koraku $2, 3, \dots, t$ uporabimo funkcijo f
- Delna rešitev: funkcije aktivacije, ki ne zmanjšujejo vrednosti argumentov, npr. ReLU

Krčenje **gradientov** zaradi produkta v formuli za vzvratno razširjanje.

Prava rešitev druge in tretje težave

Arhitekturi *NM*, ki omogočata vzdrževanje dolgoročnega spomina

- LSTM, *Long Short-Term Memory*, dolgi kratkoročni spomin
- GRU, *Gated Recurrent Unit*, rekurentna enota z vrati

Spoznali jih bomo pri naprednem strojnem učenju.

Znani algoritmi in implementacije

Prvi prispevek (Rosenblatt 1958)

Iterativno pravilo za klasifikacijo z enostavnim perceptronom.

Pomembni nadgradnji

- Vzvratno razširjanje napake, VRN (Werbos 1982)
- Rekurentne nevronske mreže, VRN skozi čas (Werbos 1990)

Globoko učenje (Hinton in ost. 2006)

- Knjižnica TensorFlow, nato še Keras in **PyTorch**
- V R-ju na voljo paket torch, a tudi tensorflow